

# RepoChat — Handoff Summary

## AWS Hosting Strategy & Current State

---

### What the app is

RepoChat is an AI-powered developer tool that lets users ask natural language questions about any public GitHub repository using a RAG (retrieval-augmented generation) pipeline.

**Tech stack:**

- Frontend: Next.js, React, TypeScript, Tailwind
  - Backend: Node.js, Express, TypeScript, OpenAI SDK, Tree-sitter, simple-git
  - Database: MongoDB Atlas (vector search)
- 

### Final AWS hosting strategy

#### Architecture overview

User → CloudFront → /api/\* → API Gateway → Lambda (Express backend)  
→ /\* → S3 (static Next.js frontend)

A single CloudFront distribution routes all traffic. No load balancer. The backend runs as a Lambda function. The frontend is a static Next.js build stored in S3.

**Estimated cost: ~\$0.55/month**

### Services

| Service              | Purpose                       | Cost       |
|----------------------|-------------------------------|------------|
| MongoDB Atlas M0     | Database + vector search      | Free       |
| ECR (1 repo)         | Backend Docker image registry | ~\$0.05/mo |
| SSM Parameter Store  | Encrypted backend secrets     | Free       |
| IAM                  | Permissions — no static keys  | Free       |
| Lambda + API Gateway | Backend Express server        | ~\$0.00/mo |
| S3                   | Static frontend files         | ~\$0.00/mo |
| CloudFront           | CDN + routing                 | ~\$0.00/mo |
| ACM                  | SSL certificate               | Free       |
| Route 53             | DNS                           | \$0.50/mo  |
| CloudWatch           | Logs                          | Free       |

---

## Key decisions made

**Lambda over ECS for backend** App Runner was deprecated (no new customers after April 30, 2026). ECS Express Mode costs ~\$16-18/month due to ALB. Lambda costs near zero at low traffic. AWS Lambda Web Adapter wraps the Express server with no application code changes.

**Static S3 over SSR for frontend** Brings cost from ~\$16/month (ECS Express Mode) to ~\$0.55/month. SSR on Lambda is possible via OpenNext but adds complexity. Migration guide exists to move to ECS Express Mode SSR when needed — estimated one afternoon of work.

**SSM Parameter Store over Secrets Manager** No rotation needed. Free vs \$0.40/secret/month for Secrets Manager.

**CloudFront eliminates load balancer** Routes `/api/*` to API Gateway and `/*` to S3 from one distribution. Saves ~\$18/month vs ALB.

**Domain: Hostinger registration, Route 53 DNS** Nameservers switched from Hostinger to Route 53 because Route 53 Alias records allow pointing the apex domain (`yourdomain.com`) at CloudFront — standard DNS CNAME cannot do this. Hostinger still owns domain registration.

---

## Backend Lambda — implementation details

### Lambda Web Adapter

The Express backend runs via AWS Lambda Web Adapter (`public.ecr.aws/awsguru/aws-lambda-adapter:0.8.4`). The adapter runs as a Lambda extension, intercepts Lambda events, and forwards them as HTTP requests to Express on port 8080. Zero application code changes required beyond port configuration.

### Dockerfile key points

- Multi-stage build: `deps` → `builder` → `runner`
- Lambda Web Adapter binary copied to `/opt/extensions/lambda-adapter`
- `PORT=8080, AWS_LWA_PORT=8080`
- `AWS_LWA_READINESS_CHECK_PATH=/api/health/check`
- `HOME=/tmp` (required — git fails without it in Lambda)
- Built with `--platform linux/amd64 --provenance=false` (required on Apple Silicon)
- Non-root user (`appuser`)
- git installed via `apk add --no-cache git` (needed for repo cloning)

### Build and push command

```
docker buildx build \  
  --platform linux/amd64 \  
  --provenance=false \  
  --
```

```
-t <account-id>.dkr.ecr.ca-central-1.amazonaws.com/repo-chat/backend:latest \
--push \
./backend
```

## Lambda function config

- Memory: 512MB
- Timeout: 3 minutes (180s) — needed for search index creation on first run
- Region: ca-central-1

## SSM parameters (ca-central-1)

```
/repo-chat/prod/REPO_STORAGE_PATH → /tmp/repoCloning
/repo-chat/prod/MONGO_URL          → mongodb+srv://...
/repo-chat/prod/LOG_LEVEL
/repo-chat/prod/OPENAI_API_KEY
/repo-chat/prod/OPENAI_CHAT_MODEL
/repo-chat/prod/OPENAI_EMBEDDING_MODEL
```

PORT is NOT in SSM — set via Dockerfile ENV only.

## IAM role

LambdaBackendExecutionRole has:

- AWSLambdaBasicExecutionRole (managed)
- AmazonEC2ContainerRegistryReadOnly (managed)
- Inline policy: ssm:GetParameter and ssm:GetParameters on  
arn:aws:ssm:ca-central-1:867344429725:parameter/repo-chat/prod/\*

## Critical fixes applied during deployment

1. **Import ordering** — appConfig from config.js must be dynamically imported AFTER loadSecrets() because static imports are hoisted and run before async code. Pattern: `const { appConfig } = await import('./config/config.js')` inside startServer() after `await loadSecrets()`.
2. **MongoDB connection** — removed `directConnection: true` (incompatible with `mongodb+srv://` Atlas connection string). Use `process.env.MONGO_URL` directly in `connectToDatabase()`, not via imported config object (config values are captured at module load time before SSM loads).
3. **PORT conflict** — PORT was in SSM as 5000, overriding Dockerfile's PORT=8080. Lambda Web Adapter expected port 8080, server was on 5000. Fixed by removing PORT from SSM.
4. **Health check method** — health route was POST, Lambda Web Adapter readiness check uses GET. Changed to `health.get('/check', ...)`.
5. **Platform flag** — must use `--platform linux/amd64 --provenance=false` on

Apple Silicon. Without `--provenance=false`, Docker creates a manifest list that Lambda rejects.

6. **HOME not set** — git requires HOME env var. Added `ENV HOME=/tmp` to Dockerfile and as Lambda environment variable.
  7. **Search index blocking startup** — `createSearchIndexes()` moved to run in background (`.catch()` pattern) so it does not block server startup or cause Lambda timeout.
  8. **Atlas IP access** — Atlas Network Access must have `0.0.0.0/0` since Lambda has no fixed IP.
- 

## Frontend — implementation details

### next.config.js

```
const nextConfig = {
  output: "export",
  images: { unoptimized: true },
};
```

### API calls

Use relative paths — both frontend and backend behind same CloudFront domain:

```
const API_URL = process.env.NEXT_PUBLIC_API_URL || '';
const res = await fetch(`${API_URL}/api/query`, ...);
```

NEXT\_PUBLIC\_API\_URL is empty in production (relative paths). Set to `http://localhost:8080` in `.env.local` for local dev.

### S3

Bucket: `repo-chat-frontend` in `ca-central-1`. Private bucket, OAC enabled via CloudFront wizard. Upload `out/` directory after `next build`.

### NEXTPUBLIC note

These vars are baked in at build time. For Docker builds pass as build arg:

```
docker build --build-arg NEXT_PUBLIC_API_URL=http://localhost:8080 -t
yourapp/frontend ./frontend
```

---

## CloudFront — current state

Distribution created via new AWS wizard (UI different from older guides). S3 origin configured with OAC. Still needs:

1. **Add API Gateway origin** — add second origin pointing at API Gateway invoke URL

2. **Add `/api/*` behavior** — route to API Gateway origin with `CachingDisabled` and `AllViewerExceptHostHeader`
  3. **Set default root object** — `index.html`
  4. **Add SPA error responses** — `403 → /index.html (200)` and `404 → /index.html (200)`
  5. **Domain** — add alternate domain names + ACM certificate once issued
- 

## ACM + Route 53 — current state

- Nameservers: switched from Hostinger to Route 53 (or in progress)
  - Domain: `tarifmohammad.com` registered in Hostinger
  - To verify Route 53 is active: run `nslookup -type=NS tarifmohammad.com` and confirm Route 53 nameservers are returned
- 

## Migration path to SSR (future)

When SSR is needed, the migration from static to ECS Express Mode SSR is one afternoon of work:

- Change output: `'export'` to output: `'standalone'` in `next.config.js`
- Add Next.js SSR Dockerfile
- Create ECS cluster + IAM roles + Express Mode service
- Swap CloudFront default `/*` origin from S3 to ECS Express Mode URL
- Add `/_next/static/*` behavior with `CachingOptimized`
- Delete S3 bucket
- Additional cost: ~\$16/month (Fargate + ALB)